

robodog

Software architecture for a 12-DOF quadruped
12 × ROBSTRIDE02 QDD actuators • 10 kg • ROS 2 Jazzy

Generated from the Onshape CAD export 600e4a0e7d58f2e2

Revision 0.1 — simulation-complete, hardware-ready

Abstract

This document specifies the first software architecture for a 12-DOF quadruped built around twelve ROBSTRIDE02 quasi-direct-drive actuators. The robot model is derived programmatically from the CAD assembly; the control stack runs at 400 Hz behind a hardware abstraction that has three implementations, two simulated and one for the real CAN bus. Everything is implemented and running in simulation. Nothing above the hardware boundary changes when the real actuators and camera are connected — that is the property the architecture is organised around, and Section 11 states exactly what remains to be done.

Contents

1	Scope and status	2
2	System overview	3
2.1	Design targets	3
2.2	Mass budget	3
2.3	Coordinate systems and conventions	4
3	Mechanical model	4
3.1	CAD to URDF pipeline	4
3.2	Kinematics	5
3.3	Knee drive topology	5
3.4	Joint limits	6
4	Actuator: ROBSTRIDE02	6
4.1	Specification and derived quantities	6
4.2	CAN interface	6
4.3	Bus budget	7
5	Software architecture	8
5.1	Packages	8
5.2	The hardware boundary	9
5.3	Interfaces	10
6	Control architecture	11
6.1	The loop	11
6.2	Impedance control	12
6.3	Gravity feed-forward and the force convention	12
6.4	Gait generation	12
6.5	Balance	13
6.6	Measured locomotion performance	13
6.7	State estimation	14

7	Safety	14
7.1	Two-tier limits	14
7.2	Design rules	14
7.3	Torque limiting under impedance control	15
7.4	Thermal model	15
7.5	Watchdog and emergency stop	15
8	Simulation architecture	16
8.1	Fidelity choices	16
8.2	Validation against the model	17
8.3	Test environment	17
9	Perception	18
10	Web GUI	19
11	Migration to hardware	19
11.1	What changes	19
11.2	Bring-up sequence	19
11.3	Known gaps	20
11.4	Planned next steps	20
12	Verification	20

1 Scope and status

What exists. Eight ROS 2 packages, 171 passing tests, a MuJoCo model generated from the same parameters as the URDF, a five-room test house, a modular web GUI, and the complete ROBSTRIDE02 CAN frame codec with 29 unit tests. The robot stands, holds pose and walks in simulation.

What does not. No hardware has been driven. The CAN backend is written but unvalidated; per-joint direction and offset are unknown until the robot is assembled and calibrated. The camera depth path is a placeholder in one respect only (how the HP60C delivers depth over USB). Estimated quantities — rotor inertia, thermal constants, joint friction — are marked as such throughout and listed in Section 11.3.

How to read this. Every quantity quoted here is generated from `robot_parameters.yaml` and `robstride02.yaml` by `tools/make_doc_facts.py`. If the CAD changes and the model is regenerated, the numbers in this document change with it.

2 System overview

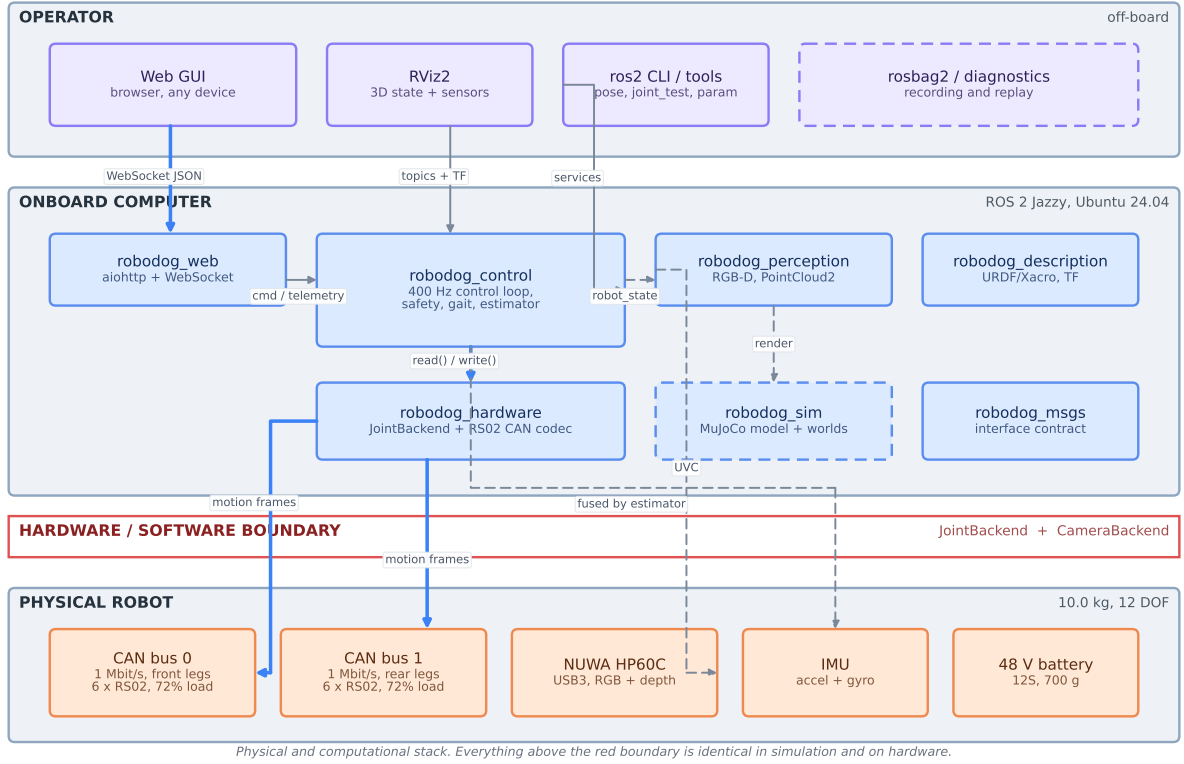


Figure 1: System architecture. The red band is the hardware/software boundary: two abstract interfaces, JointBackend and CameraBackend. Simulation and hardware differ only below it.

2.1 Design targets

Quantity	Target	Achieved
Total mass	10 kg	10 kg
Degrees of freedom	12 (HAA/HFE/KFE)	12
Nominal stance height	—	320 mm (69.7 % of leg reach)
Static torque margin	within continuous	62.5 % of 6 N·m
Control rate	—	400 Hz, sim and hardware alike
Actuator	ROBSTRIDE02	6/17 N·m cont/peak

2.2 Mass budget

The CAD assembly was drawn around RMD-X8 actuators at 760.8 g each. Substituting ROBSTRIDE02 at 380 g removes 4.57 kg and lands the structure at 8.892 kg, leaving 1.04 kg for electronics and 0.068 kg for the camera.

Item	Mass (kg)	Note
CAD structure	4.332	body, brackets, legs, bearings
12 × ROBSTRIDE02	4.56	380 g each
Battery, 12S LiPo	0.700	48 V nominal
Onboard computer	0.200	
Power distribution and wiring	0.130	
IMU	0.010	
Camera (NUWA HP60C)	0.068	separate link
Total	10	

Per-link masses after aggregation: base 3.934 kg, hip 624.5 g, thigh 703.4 g, calf 171.7 g. The whole-robot centre of mass sits -3.51 mm from the body centre longitudinally.

2.3 Coordinate systems and conventions

Frames. REP-103 throughout: x forward, y left, z up. `base_link` is at the body geometric centre. `odom` is the world frame for the estimated base pose and the test-world markers. Camera optical frames follow the ROS optical convention (z forward, x right, y down), related to their body frames by $\text{rpy} = (-\pi/2, 0, -\pi/2)$.

Joint naming. `{FL,FR,RL,RR}_{haa,hfe,kfe}_joint`. HAA is hip abduction/adduction, HFE hip flexion/extension, KFE the knee.

Decision — Joint axes are not mirrored between left and right

Every HAA axis is $+x$ and every HFE and KFE axis is $+y$, expressed in `base_link` orientation, for all four legs. The alternative — mirroring the right side so that “positive” means abduction everywhere — was rejected because it makes the URDF axes differ per leg and turns every sign question into a per-leg question.

The consequence must be remembered: **positive HAA abducts the left legs and adducts the right legs**. Sim-to-real then reduces to one sign per joint in `robstride_bus.yaml`, not a frame convention.

Zero and sign. All joints at zero puts the legs straight down, feet at $(\pm 0.221, \pm 0.1445, -0.430)$ m. Positive HFE swings the thigh backward. Negative KFE folds the knee backward; $\text{KFE} = 0$ is fully extended and the upper limit, so the knee cannot hyperextend.

Decision — Named poses are specified as a base height, not joint angles

Each foot hangs vertically below its own HFE axis. That one rule yields, at once: identical joint angles on all four legs despite the non-mirrored axes; a support polygon of 442×289 mm centred within -3.51 mm of the centre of mass; and the lowest peak joint torque of any centred foot placement.

Placing the feet under the *HAA* axes instead would need different front and rear angles, because the HFE sits 60 mm outboard of the HAA, and would cost 73 % of the continuous rating on the rear knees instead of 62.5 %.

3 Mechanical model

3.1 CAD to URDF pipeline

The Onshape export is a flat tree of 85 part-links with 12 revolute joints, saved at an arbitrary mate pose. `tools/cad_to_model.py` turns it into a canonical 13-body model:

1. forward-kinematics every part into the root frame;
2. cut the tree at the 12 revolute joints, giving 13 rigid bodies;
3. recover the pose-invariant leg geometry (link lengths, axis offsets);
4. solve the CAD mate angles against the canonical zero definition;
5. re-express every visual mesh and inertia tensor in canonical body frames;
6. substitute ROBSTRIDE02 mass properties for the CAD’s RMD-X8 placeholders.

The canonical chain reproduces the CAD joint positions to within $9.9\ \mu\text{m}$, and the four legs agree with each other to under $2\ \mu\text{m}$ — the CAD is exactly symmetric, and the residual is mesh float precision.

Decision — The URDF is generated, never hand-edited

`robot_parameters.yaml` is the single source of truth. The Xacro, the MuJoCo MJCF, the inverse kinematics and this document all read it. A hand-maintained URDF drifts from the CAD, and a drift between the model the physics uses and the model the IK uses presents as a control bug that is very hard to attribute.

3.2 Kinematics

Parameter	Value	Meaning
HAA position	$\pm 161, \pm 60\ \text{mm}$	from <code>base_link</code>
HAA $\rightarrow$ HFE	60 mm longitudinal, 16 mm radial	
Thigh length L_1	213 mm	HFE to KFE, perpendicular
Thigh lateral	68.5 mm	along the HFE axis
Shank length L_2	217.32 mm	KFE to foot centre
Leg-plane offset	84.5 mm	constant, = radial + lateral
Maximum reach	430.32 mm	$L_1 + L_2$
Foot sphere	20 mm radius	contact geometry

Because rotation about y leaves y unchanged, the lateral offset collapses to a constant. That is what makes the inverse kinematics closed-form: the abduction angle is fixed by the (y, z) projection alone, and what remains is a planar two-link problem. Forward kinematics from the HAA joint frame:

$$p = R_x(q_0)(d_1 + R_y(q_1)(d_2 + R_y(q_2) d_3))$$

with $d_1 = (s_x \text{ hfe}_{dx}, s_y \text{ hfe}_{dr}, 0)$, $d_2 = (0, s_y \text{ lat}, -L_1)$, $d_3 = (0, 0, -L_2)$.

The IK returns only the knee-backward branch ($q_2 \leq 0$); the mirror solution exists mathematically but the robot cannot reach it. Round-tripping FK through IK over the full joint range gives a worst-case position error below $10^{-9}\ \text{m}$, and the analytic Jacobian matches central differences to 10^{-6} .

3.3 Knee drive topology

The knee actuator is mounted *proximally* on the thigh — 48 mm from the HFE axis, 179 mm from the knee — and drives the joint through a 1:1 belt.

Decision — Proximal knee actuator, modelled as a simple revolute joint

This is the standard low-distal-inertia arrangement. Its effect is measurable in the generated model: the thigh’s centre of mass sits only 27 mm below the hip, and the calf masses just 171.7 g. Because the motor rotates *with* the thigh, the knee angle is not kinematically coupled to the hip angle, so the URDF needs no mimic joint or transmission — the belt ratio is a parameter that is currently 1.0. Should a non-unity ratio be fitted, it enters in one place: the joint-to-motor map.

3.4 Joint limits

Joint	Lower (rad)	Upper (rad)	Rationale
HAA	-0.8	0.8	$\pm 45.8^\circ$ abduction envelope
HFE	-1.4	2.4	thigh forward to folded
KFE	-2.6	0	no hyperextension; folds backward only

The CAD's own joint limits are not used

The Onshape export carries mate limits that are modelling artefacts — for example `dof_f10` spans $[-0.704, 0.866]$, asymmetric and different on every leg. They describe how the assembly was posed, not what the mechanism can do. The limits above are design values chosen from the mechanical envelope and should be refined once self-collision checking runs against the real geometry.

4 Actuator: ROBSTRIDE02

4.1 Specification and derived quantities

Quantity	Source	Value
Mass	spec	$380 \text{ g} \pm 3 \text{ g}$
Envelope	spec	$78.5 \text{ mm dia} \times 41.5 \text{ mm}$
Gear ratio	spec	7.75:1, machined steel
Rated / peak torque	spec	$6 / 17 \text{ N}\cdot\text{m}$
Rated speed	spec	$100 \text{ rpm} = 10.47 \text{ rad/s}$
No-load speed	spec	$410 \text{ rpm} = 42.94 \text{ rad/s}$
Torque constant K_t	spec	$1.22 \text{ N}\cdot\text{m}/\text{A}_{\text{rms}}$
Line resistance	spec	0.58Ω (phase 0.29Ω)
Voltage	spec	48 V rated, $24\text{--}60 \text{ V}$
Encoders	spec	$2 \times 14\text{-bit}$ absolute magnetic
Output resolution	derived	$3.835 \times 10^{-4} \text{ rad}$
Rated phase current	derived	$4.95 \text{ A}_{\text{rms}}$
Copper loss at rating	derived	21.3 W
Reflected rotor inertia	estimate	$4.805 \times 10^{-3} \text{ kg}\cdot\text{m}^2$

K_t is output-referred. Dividing the rated torque by the torque constant gives $6/1.22 = 4.92 \text{ A}_{\text{rms}}$, which matches the rated phase current of $7 \text{ A peak} / \sqrt{2} = 4.95 \text{ A}_{\text{rms}}$. The constant is therefore quoted *after* the 7.75:1 reduction. This matters: it makes the torque-to-current conversion used by the thermal observer a single multiplication, with no gear ratio in it.

Reflected inertia dominates the distal link. With $J_{\text{rotor}} \approx 8 \times 10^{-5} \text{ kg m}^2$ estimated for a 28-pole outrunner, the inertia seen at the output is $J_{\text{rotor}} \times 7.75^2 = 4.805 \times 10^{-3} \text{ kg m}^2$ — larger than the calf's own inertia about the knee. Omitting it from the simulation would make the legs roughly four times too easy to accelerate.

4.2 CAN interface

Extended 29-bit identifiers at 1 Mbit/s. The identifier carries `[28:24] comm type | [23:8] data | [7:0] target`. The motion-control frame (type 1) is the MIT-style impedance command: target angle, velocity, k_p and k_d occupy the eight payload bytes, and the feed-forward torque rides in the identifier's data field because the payload is full.

Decision — The command interface is the actuator’s native impedance frame

`JointCommand` carries (position, velocity, effort, k_p , k_d) — exactly one CAN frame. A narrower interface, such as position only, would force the host to close a loop the motor already closes in firmware at tens of kHz. The host sets *setpoints*; it is not the servo.

4.3 Bus budget

An extended-ID frame with eight data bytes is 131 bits nominal and up to 159 with worst-case stuffing; 150 bits is the design figure. Each joint needs one command and one feedback frame per cycle.

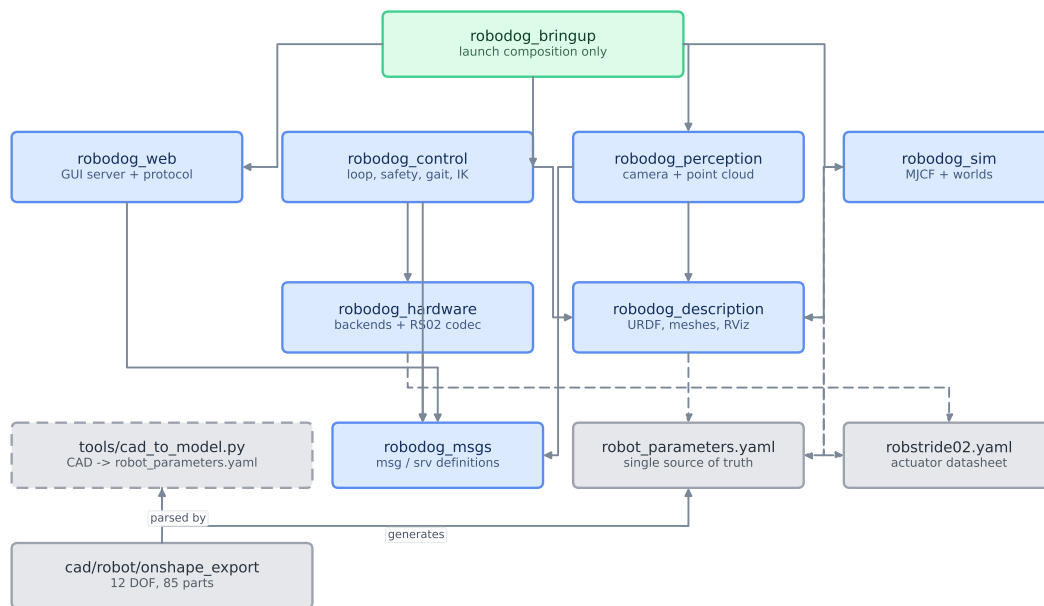
Buses	Motors/bus	Rate	Load	Verdict
1	12	250 Hz	90 %	too tight, no margin
1	12	400 Hz	144 %	impossible
2	6	400 Hz	72 %	selected
2	6	500 Hz	90 %	rejected, no margin
3	4	500 Hz	60 %	upgrade path

Decision — Two buses at 400 Hz, split front/rear

400 Hz is not a compromise on control quality: because the RS02 closes its position and velocity loop in firmware, the host rate is a *setpoint update* rate. The same rate is used in simulation so that gains tuned in MuJoCo transfer without change.

The split is front/rear rather than left/right so that each bus carries a complete leg *pair*: a single bus failure leaves the robot in a recognisable posture rather than a diagonal collapse. `RobStride02Backend.check_bus_budget()` refuses any configuration above 80 % at start-up.

5 Software architecture



ROS 2 package graph. Arrows point from dependant to dependency; no package above depends on a simulation package.

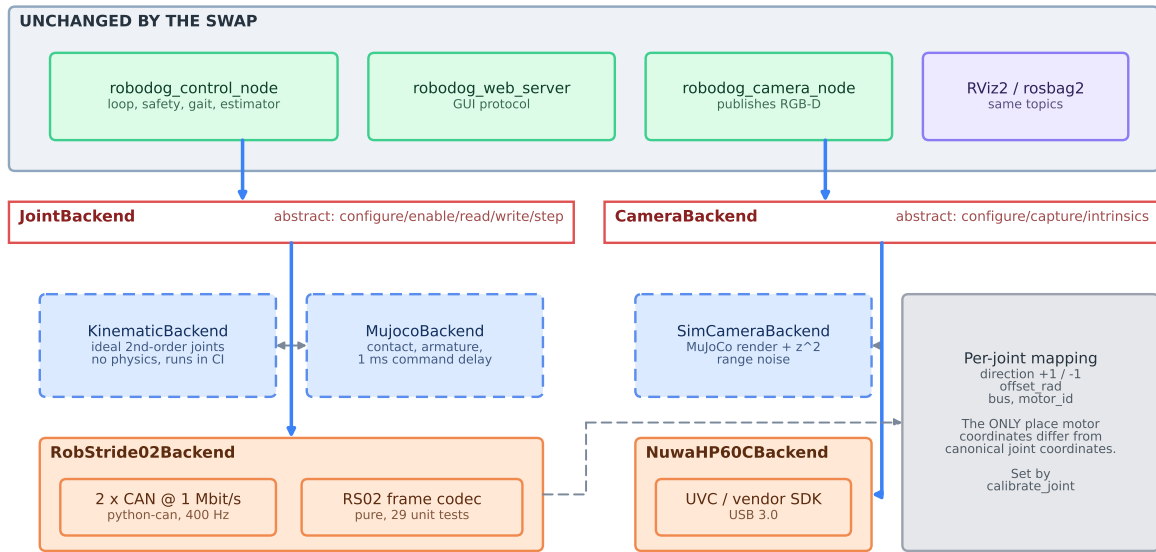
Figure 2: ROS 2 package graph. Arrows point from dependant to dependency. No package above the hardware layer depends on a simulation package.

5.1 Packages

Package	Build	Responsibility
robodog_msgs	ament_cmake	message and service contract
robodog_description	ament_cmake	URDF/Xacro, meshes, RViz, actuator data
robodog_hardware	ament_python	JointBackend + RS02 CAN codec
robodog_control	ament_python	control loop, safety, gait, IK, estimator
robodog_sim	ament_python	MJCF generator, test worlds, markers
robodog_perception	ament_python	CameraBackend, depth, point cloud
robodog_web	ament_python	GUI server and protocol
robodog_bringup	ament_cmake	launch composition only

`robodog_msgs` depends on nothing but `std_msgs` and `geometry_msgs`: it is the contract between layers and must not acquire an implementation dependency.

5.2 The hardware boundary



Replacing simulation with hardware is a launch argument. Nothing above the boundary changes.

Figure 3: Everything above the boundary is unchanged by the swap to hardware. The per-joint direction and offset map is the only place where motor coordinates and canonical joint coordinates differ.

JointBackend is a six-method abstract class:

```
class JointBackend(ABC):
    name: str; is_simulation: bool
    def configure(self) -> None           # open devices / load model
    def enable(self, mask) -> None         # energise
    def disable(self, mask) -> None        # de-energise, back-driveable
    def read(self) -> JointState            # pos, vel, effort, temperature, faults
    def write(self, cmd: JointCommand)     # already safety-clamped
    def step(self, dt) -> None              # advance a simulator; no-op on
        hardware                           hardware
    def base_state(self) -> BaseState | None # None on real hardware
```

Decision — base_state() returns None on the real robot

A simulator knows the floating-base state exactly; the real robot has no sensor for absolute base pose. Rather than inventing one, the contract makes the absence explicit and forces every consumer to handle it. The state estimator fills the gap from the IMU and leg kinematics, and marks its output `ground_truth = False`. Returning a plausible-looking zero would hide the difference that matters most when a controller tuned in simulation is first run on hardware.

Three implementations exist. `KinematicBackend` models each joint as an independent second-order system with the reflected rotor inertia — no physics engine, no contact, no gravity — so the entire stack above the boundary is runnable and testable with nothing installed. `MujocoBackend` adds rigid-body dynamics and contact. `RobStride02Backend` drives the real motors.

5.3 Interfaces

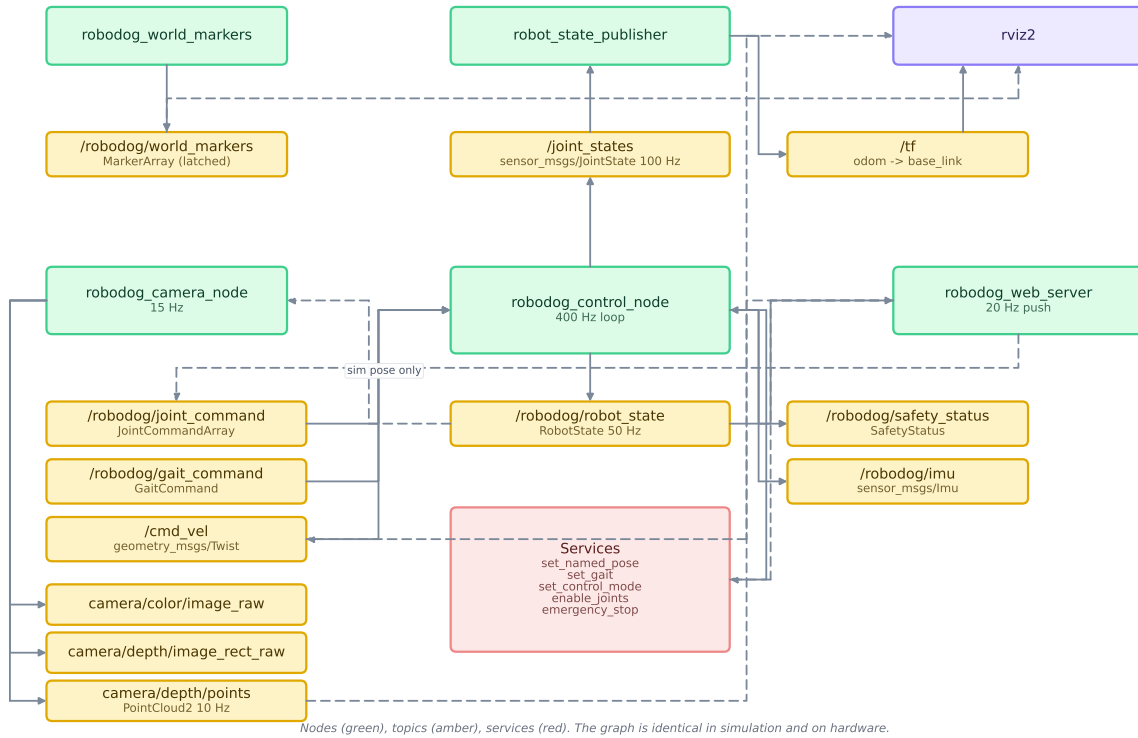


Figure 4: Node and topic graph. Identical in simulation and on hardware; only the implementations behind the two backends differ.

Topic	Type	Rate	Purpose
/joint_states	JointState	100	TF, RViz, standard tooling
/robodog/robot_state	RobotState	50	aggregate telemetry
/robodog/safety_status	SafetyStatus	50	limits, faults, loop health
/robodog/imu	Imu	50	attitude and rates
/robodog/joint_command	JointCommandArray	—	direct joint control
/robodog/gait_command	GaitCommand	—	gait selection
/cmd_vel	Twist	—	body velocity
/robodog/world_markers	MarkerArray	latched	test environment
camera/color/image_raw	Image (rgb8)	15	colour
camera/depth/image_rect_raw	Image (16UC1)	15	depth, millimetres
camera/depth/points	PointCloud2	10	range-filtered cloud

Services: set_named_pose, set_gait, set_control_mode, enable_joints, emergency_stop.

Decision — One aggregate telemetry message

RobotState carries all twelve joints, the four feet, the base estimate, controller state, safety state and simulation state in one message. A consumer therefore never has to time-align several topics to render a consistent picture. /joint_states is published separately and at a different rate because the wider ROS ecosystem expects it.

6 Control architecture

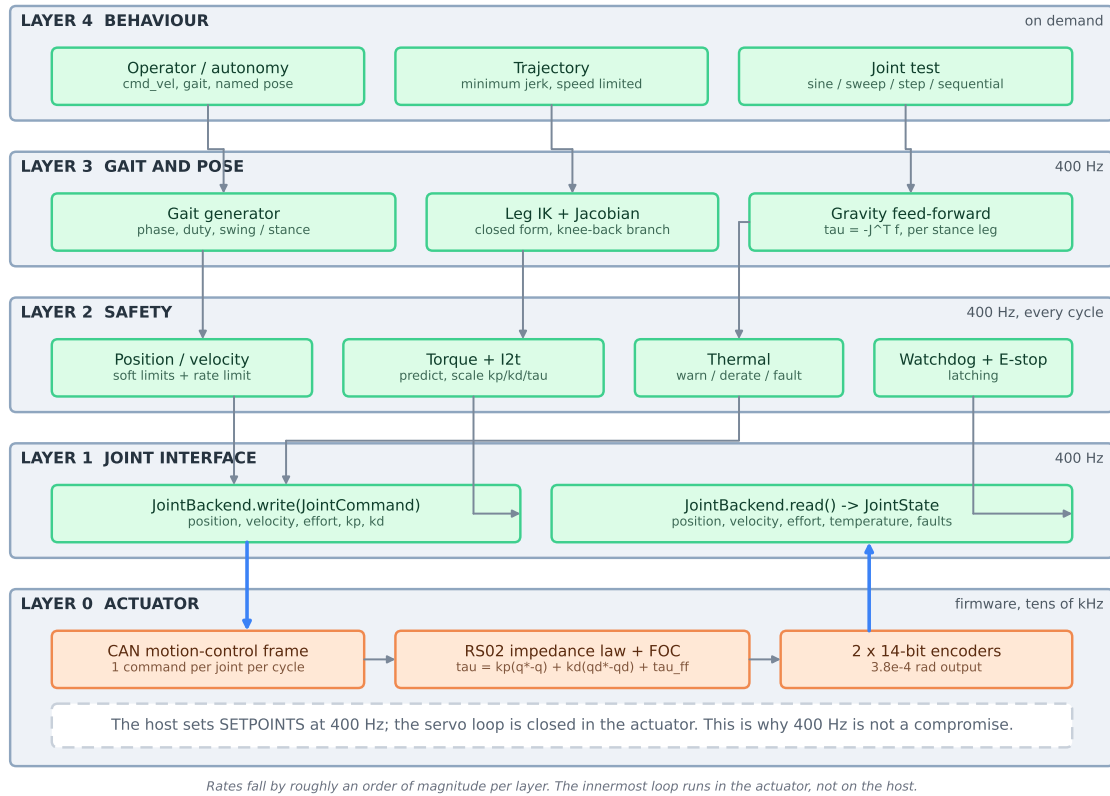


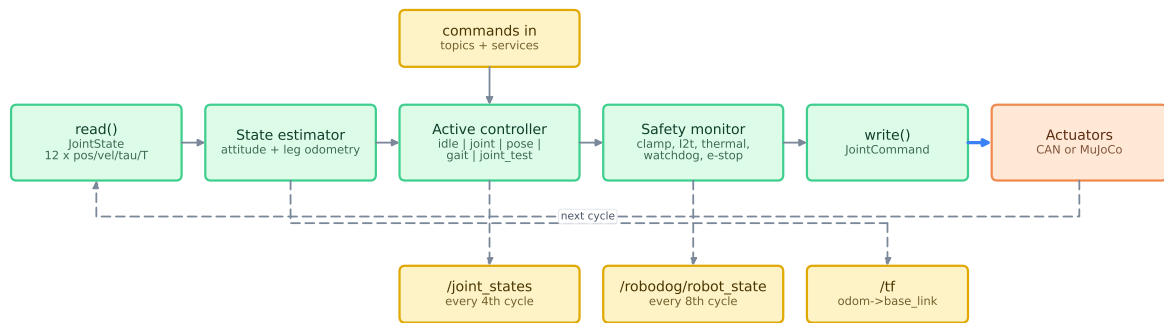
Figure 5: Control layers. Rates fall by roughly an order of magnitude per layer, and the innermost loop is not on the host at all.

6.1 The loop

One thread, 400 Hz, in `robodog_control_node`:

`read()` → estimate → active controller → safety → `write()` → `step()`

ROS callbacks run on a separate executor and only ever set fields the loop reads; nothing in a callback touches the backend. Publishing is decimated — `RobotState` at 50 Hz, `/joint_states` at 100 Hz — because publishing telemetry at 400 Hz would cost more than the control it reports on.



One 2.5 ms cycle. The safety monitor is unconditional: nothing reaches the actuators without passing it.

Figure 6: One 400 Hz cycle. The safety monitor is unconditional.

Decision — Controllers are selected, not composed

Exactly one of {idle, joint, pose, gait, joint_test} drives the joints at any moment. Making that exclusive by construction is worth more than the flexibility of a controller chain, because two things writing to the same actuator is a failure mode with no good recovery.

6.2 Impedance control

The actuator evaluates

$$\tau = k_p (q^* - q) + k_d (\dot{q}^* - \dot{q}) + \tau_{ff}$$

in firmware. Effective joint inertia is the reflected rotor inertia (4.805×10^{-3}) plus roughly 2×10^{-3} of link, so $J \approx 7 \times 10^{-3} \text{ kg m}^2$; $k_p = 120$ then gives $\omega_n = \sqrt{k_p/J} \approx 131 \text{ rad/s}$ (21 Hz) and a critical damping of $k_d = 2\sqrt{k_p J} = 1.83$.

Context	k_p	k_d	Rationale
Named pose, joint test	80	2.0	moderate, unloaded
Stance leg	120	2.5	stiff enough to hold the body up
Swing leg	45	1.2	compliant: an unexpected contact during swing must not lever the robot over

6.3 Gravity feed-forward and the force convention

Each stance leg is fed the torque that holds its share of the body weight. The sign convention is stated once and used everywhere:

f is the force the *environment* applies to the foot (the ground reaction, $+z$ in stance), so

$$\tau_{\text{actuator}} = -J^T f, \quad f = -(J^T)^{-1} \tau_{\text{actuator}}.$$

This sign is not cosmetic

An inverted feed-forward makes a stance leg push the body *down*. Measured in MuJoCo: with correct feed-forward the body sags 0.31 mm below the commanded 320 mm; without feed-forward it sags 5.34 mm; with the sign inverted it sags 11.95 mm. Magnitude-only tests pass in all three cases, which is why `test_kinematics.py` asserts the *direction* of the knee torque explicitly.

6.4 Gait generation

A single normalised cycle phase advances at the step frequency. Each leg has a fixed offset and is in stance for `duty_factor` of its period.

Gait	Offsets	Duty	Freq (Hz)	Character
stand	—	1.00	0.0	all feet planted
walk	0, 0.50, 0.75, 0.25	0.75	1.2	statically stable, one foot up
trot	0, 0.50, 0.50, 0	0.50	2.2	diagonal pairs; default travel
pace	0, 0.50, 0, 0.50	0.50	2.0	lateral pairs; rolls this narrow body
bound	0, 0, 0.50, 0.50	0.45	2.6	front then rear; most demanding

Swing horizontal position follows $s - \sin(2\pi s)/2\pi$, whose derivative $1 - \cos(2\pi s)$ vanishes at both ends, so the foot lifts off and touches down with no horizontal velocity relative to the ground. Height follows $\sin^2(\pi s)$, likewise zero-slope at both ends, with a small `touchdown_depth` subtracted late in swing so the foot seeks the ground rather than hovering over an uneven floor. Stride length is not configured: it follows from the commanded velocity and the stance duration, clamped to 45 % of leg reach.

6.5 Balance

A foot-trajectory generator alone does not walk. With the open-loop gait and no balance term, a 0.3 m/s trot in MuJoCo reached 25° of body tilt, saturated the actuators at the 17 N·m peak and travelled 28 mm in six seconds. The feet were following their commanded paths; the body was not being held up. `balance.py` adds the two things that were missing.

Force distribution. A PD controller on height, roll, pitch, yaw rate and horizontal velocity produces a desired body wrench, and the stance foot forces are solved to reproduce it:

$$\sum_i f_i = F, \quad \sum_i r_i \times f_i = M$$

Six equations in $3n_{\text{stance}}$ unknowns — exactly determined for a trot, under-determined for a walk or a full stance — solved by least squares, then projected into what a contact can deliver: $f_z \geq 0$ and $\|f_{xy}\| \leq \mu f_z$.

Decision — A stance leg is force controlled, not position controlled

This was measured the hard way. With `stance_kp` at 120 the stance legs showed 0.17 rad of tracking error and 20 N·m of pure disagreement with reality: the commanded foot position was advancing at the *commanded* body velocity while the foot was physically planted, so the leg was dragging the robot through its own contact. Two changes follow. The stance foot now translates at a blend of the *measured* and commanded velocity (`stance_sweep_gain`), and the placement offset is latched at touchdown so the command cannot move under a planted foot. Stiffness is reduced to 90: raising it turns the leg into a spring that anchors the body to its own foot and cancels the propulsion force.

Foot placement. Attitude control cannot regulate velocity — a legged robot steers by where it puts its feet. The Raibert heuristic places the landing point at $v T_{\text{stance}}/2 + k_v(v - v^*)$.

6.6 Measured locomotion performance

Gait	v_x^* (m/s)	travelled (m in 6 s)	height (m)	peak tilt (deg)	Verdict
stand	0.00	−0.003	0.320	0.1	solid; 5.5 N·m peak, no clamping
trot	0.30	−0.241	0.320	3.0	attitude held; steps in place
trot	0.50	−0.292	0.311	10.4	attitude held; steps in place
walk	0.15	+0.528	0.256	17.0	moves forward; attitude marginal
walk	0.30	−0.433	0.119	125	falls
bound	0.30	−0.178	0.240	14.7	marginal

Locomotion does not yet track commanded velocity

Standing is solid and the balance layer regulates attitude and height well during stepping — a 0.3 m/s trot holds 3° of tilt at the commanded 320 mm, against 25° without it. But the robot steps in place rather than travelling: trot drifts backward at about 0.04 m/s, and only the slow walk makes forward progress, at the cost of attitude.

The force-to-motion path is verified correct in isolation — commanding +30 N of ground reaction with all four feet planted moves the robot forward 0.52 m, and −30 N moves it backward — so this is a tuning and timing problem, not a sign or architecture error. What it needs is work that only makes sense against identified dynamics: co-tuning stance compliance, swing timing and placement gains, contact detection instead of a fixed schedule, and slip handling. That is deliberately outside the scope of a first architecture, and none of it changes an interface.

6.7 State estimation

Decision — A complementary filter and leg odometry, not an EKF

Attitude comes from blending gyro integration with the gravity direction, with the gravity term weighted by how closely the measured acceleration magnitude resembles g — during a hard push-off it does not. Height and velocity come from treating each foot in contact as a temporary fixed point.

An EKF was rejected *for now* because until the robot exists its process and measurement covariances would be invented numbers. This is honest, debuggable and sufficient to stand, walk and visualise. Yaw is not observable from an IMU without a magnetometer and is flagged as drifting.

7 Safety

7.1 Two-tier limits

Quantity	Hard (URDF)	Operational	Enforced by
Torque	17 N·m	6 N·m continuous	I^2t integrator
Velocity	42.94 rad/s	20 rad/s	clamp
Position	joint end stops	end stops inset 0.05 rad	clamp + rate limit
Winding temperature	—	70/85/100 °C	derate, then latch

The URDF carries what the hardware can physically deliver; the operational tier is what the robot is allowed to use.

7.2 Design rules

1. **It runs in simulation too.** The monitor executes on every cycle regardless of backend. A safety layer that only activates on hardware has never been tested by the time it matters.
2. **It is a pure function.** (measurement, request, state) → (allowed command, status). No ROS, no I/O, no threads — which is what makes the whole envelope testable. `test_safety.py` is the specification.
3. **It clamps rather than rejects.** Dropping a cycle mid-stride is more dangerous than saturating one.
4. **Protective faults latch.** Clearing requires the condition to have gone away *and* an explicit operator action.

7.3 Torque limiting under impedance control

Decision — Scale the whole (k_p, k_d, τ_{ff}) triple

The motor computes $k_p e + k_d \dot{e} + \tau_{ff}$ itself, so clamping only the **effort** field would not limit the torque at all. The monitor predicts the resulting torque and, if it exceeds the budget, scales all three terms by the same factor — limiting the magnitude while preserving the commanded impedance direction.

The budget is the continuous rating normally, rising to peak while $I^2 t$ credit remains. The integrator charges as $(\tau/\tau_{cont})^2 - 1$ above the continuous rating, so twice the continuous torque exhausts the budget four times faster, and discharges when below.

7.4 Thermal model

$$P_{cu} = 3I_{rms}^2 R_\phi, \quad I_{rms} = \tau/K_t, \quad C_{th} \dot{T} = P_{cu} - \frac{T - T_{amb}}{R_{th}}$$

with $R_{th} = 2.84 \text{ K/W}$ and $C_{th} = 190 \text{ J/K}$, giving a time constant of 9 minutes. It runs in two roles: in simulation it synthesises a plausible temperature, because MuJoCo has no concept of copper loss and the safety architecture must be exercised; on the real robot it runs as an *observer* alongside the reported temperature, and the monitor takes the maximum of the two. The motor's own sensor sits on the driver board and lags the winding by tens of seconds — the observer is what catches a stalled leg pushing against an obstacle.

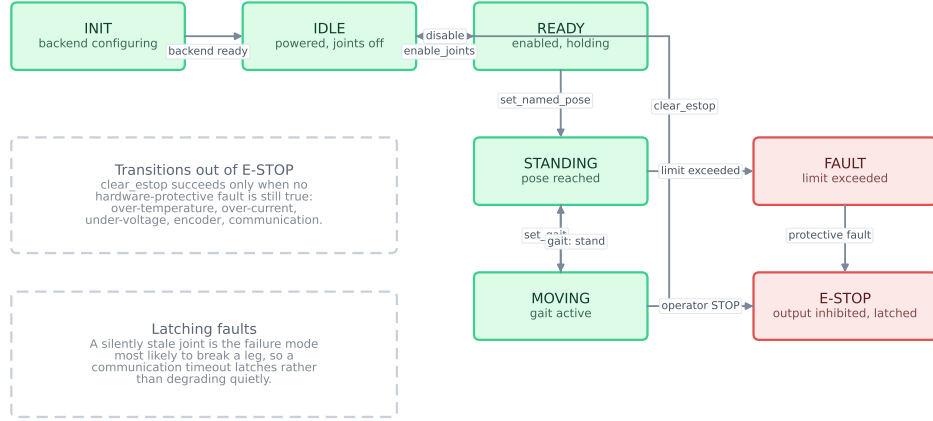
Between 85 and 100 °C the torque budget is scaled down linearly rather than cut at a threshold, because cutting output abruptly would drop the robot.

7.5 Watchdog and emergency stop

Decision — A stale command holds position; it does not go limp

Losing torque mid-stance drops a 10 kg robot. The watchdog substitutes an impedance command that holds the measured position at moderate gain. If a joint is measured outside its soft range, the hold target is the limit, so the watchdog recovers the envelope gently rather than freezing outside it.

A communication timeout on any joint latches an e-stop: a silently stale joint is the failure mode most likely to break a leg.



FAULT and E-STOP are reachable from every state. Leaving E-STOP requires the cause to have cleared.

Figure 7: Robot states. FAULT and E-STOP are reachable from everywhere; leaving E-STOP requires the cause to have cleared.

8 Simulation architecture

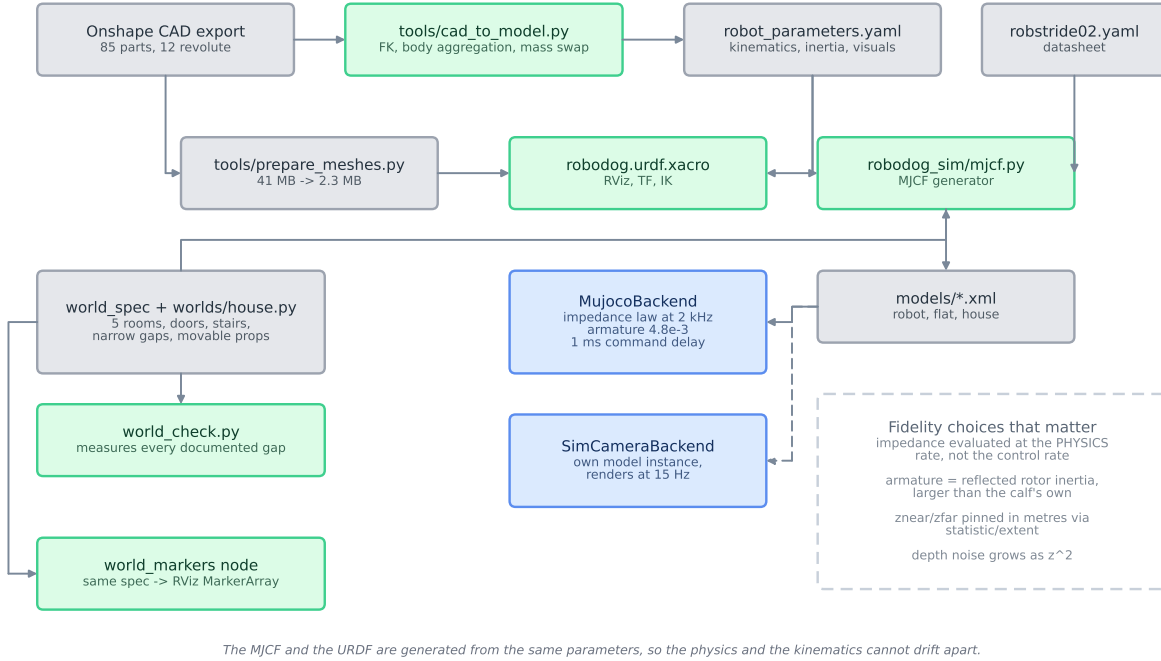


Figure 8: The MJCF and the URDF are generated from the same parameters, so the physics and the kinematics cannot drift apart.

8.1 Fidelity choices

These are not convenience settings; each is here because getting it wrong yields a controller that works in simulation and fails on the robot.

1. **The impedance law is evaluated at the physics rate**, not the control rate. The real RS02 closes its loop in firmware at tens of kHz; holding a 400 Hz zero-order hold on $k_p(q^* - q)$

would make the simulated joints noticeably softer than the hardware and would tune gains that are wrong on the robot. Only the *setpoint* is held.

2. **armature carries the reflected rotor inertia** ($4.805 \times 10^{-3} \text{ kg}\cdot\text{m}^2$), which is larger than the calf’s own inertia about the knee. Omitting it is not a small error.
3. **Command latency is injected** — two physics steps, about 1 ms, the order of a 1 Mbit/s bus at 400 Hz. Controllers tuned against a zero-latency plant are optimistic in the wrong direction.
4. **Capsules, not cylinders**, for the leg segments. URDF has no capsule primitive; MuJoCo does, and capsule contact is far better conditioned than the flat-ended cylinder a URDF must approximate it with.
5. **Timestep 0.5 ms** (2 kHz), exactly five substeps per control cycle. A non-integer ratio would make the effective command latency jitter.

Not modelled: gearbox backlash, belt compliance in the knee drive, current-loop bandwidth, and CAN jitter.

8.2 Validation against the model

Standing in MuJoCo with gravity feed-forward, measured:

Quantity	Predicted	Measured in MuJoCo
Total mass	10 kg	10 kg
Ground reaction, all feet	98.10 N	98.10 N
Front load share	47.2 %	47.0 %
Body height	320 mm	320.31 mm
Peak joint torque	62.5 % of continuous	64.6 %

The front/rear split follows from the whole-robot centre of mass sitting 12.3 mm behind the body centre in the stance pose; agreement to 0.2 % confirms that the inertia aggregation, the MJCF generation and the contact model are mutually consistent.

8.3 Test environment

Decision — One world description, two renderers

`world_spec.py` is consumed by the MJCF builder and by the RViz marker publisher. What the operator sees in RViz is geometrically what the robot collides with in MuJoCo; two hand-maintained files would drift, and the drift would look like a perception bug.

The five-room house is 12×9 m and contains every indoor locomotion problem the robot must solve, each with enough approach room to be attempted in isolation:

Feature	Dimension	Against
Doorways	0.90 m, 20 mm sill	body width 0.24 m
Narrow gaps	0.45 / 0.43 / 0.60 m	foot span 0.289 m
Stairs	4×80 mm; 3×120 mm	realistic and stretch
Ramp	12°	friction limit at $\mu = 0.9$
Under-table	0.36–0.46 m clearance	stance height 320 mm
Debris field	mixed 40–110 mm blocks	foothold selection
Movable props	4 free bodies	contact recovery

`world_check.py` measures every one of those clearances from the spec, and `test_world.py` fails if an edit turns a designed challenge into a wall or an open field. It also rejects any overlapping geometry, because MuJoCo resolves interpenetration by pushing bodies apart at the first step — silently moving the test environment out from under the experiment.

9 Perception

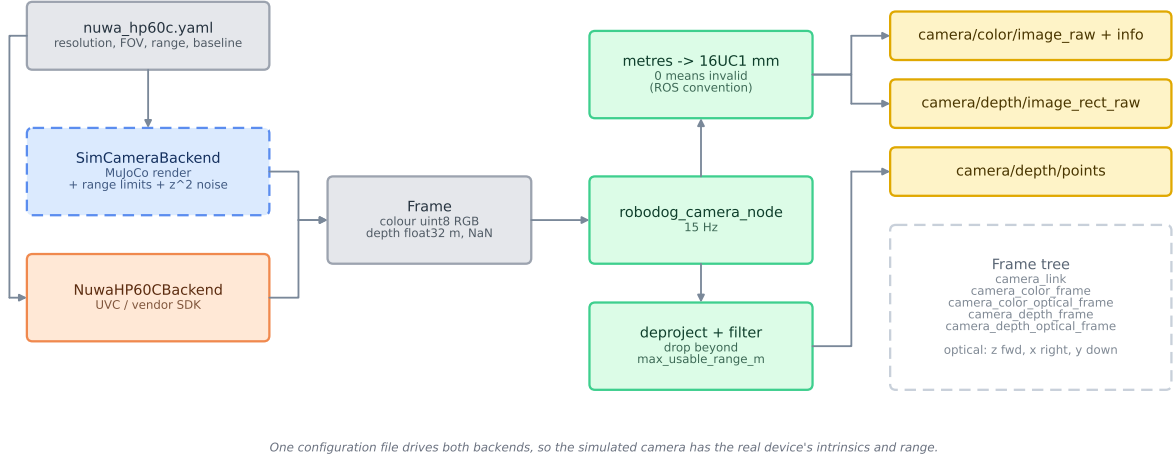


Figure 9: One configuration file drives both backends, so the simulated camera has the real device's intrinsics and range.

The Yahboom NUWA HP60C is mounted on the body's front face on a bracket at $z = 175$ mm, pitched 0.26 rad down.

Decision — The camera mount height is set by self-occlusion

The front hip assemblies occupy roughly $x = 0.11 \dots 0.26$ m and reach $z = +0.05$ m, leaving only a 42 mm slot between them on the centreline. Rendering the simulated depth image from $z = 20$ mm put **95 %** of the frame on the robot's own legs at 74 mm; $z = 90$ mm still left 48 %. Clearing the hip corner with the bottom edge of the frustum requires $z > 155$ mm, so the camera sits on a mast — as it does on every production quadruped. At 175 mm the self-view is 0 % and 98.5 % of the depth image is usable.

Decision — The simulated camera loads its own MuJoCo model

The physics instance lives in the control node's backend. Rather than reach into it, the camera backend loads the same scene read-only and drives it from the published robot state. The camera is then a separate node with a separate lifecycle — exactly as the real USB camera will be — so the node graph does not change on hardware, rendering never steals time from the 400 Hz loop, and a camera crash cannot take the control loop down.

Depth crosses the backend boundary as float32 metres with NaN for invalid, because that representation can express “no reading”; the conversion to the ROS 16UC1-millimetre convention (where 0 means invalid) happens once, at the publisher. Stereo range error grows as z^2 , so the simulated camera applies $\sigma_z = z^2 \sigma_d / (fb)$ — about 4 mm at 1 m and 140 mm at 6 m — and the point cloud is filtered to `max_usable_range_m` rather than the datasheet maximum.

Camera parameters are placeholders

Resolution, field of view, range and baseline in `config/nuwa_hp60c.yaml` are plausible for a device of this class but have *not* been read from the datasheet; each is marked [VERIFY]. The architecture does not depend on them — they are configuration, and both backends read the same file. How the HP60C delivers depth over USB (a second UVC stream, a side-by-side frame, or only through the vendor SDK) is the one thing that cannot be determined without the device, and is marked HW-CHECK in `backends/nuwa_hp60c.py`.

10 Web GUI

A single process holds an rclpy node and an aiohttp server; rclpy spins in a background thread while aiohttp owns the asyncio loop. They meet at two dictionaries: the ROS callbacks write the latest state, the web tasks read it. No queue — a GUI only ever wants the newest value, and buffering telemetry for a browser that fell behind is how a monitoring tool ends up showing the past.

Decision — A purpose-built protocol rather than rosbridge

rosbridge would re-serialise every field at the full publish rate and expose the whole graph to the browser. This speaks a small versioned JSON protocol, decimated to 20 Hz, and exposes exactly the commands the GUI is allowed to send. Less code, less bandwidth, and a much smaller surface from a page that can drive a robot.

The browser fetches `/api/config` and builds itself from it: adding, removing or reordering a panel is a change to `config/web.yaml` alone. Panels cover robot state, controller state, per-joint telemetry with tracking error and load bars, foot contact and forces, safety and thermal state, simulation state and real-time factor, an event log, camera streams, pose and gait commands, and emergency stop.

Operator limits are configuration, not constants in JavaScript: maximum commanded velocity, gait confirmation, and joint jogging — which is disabled by default because it bypasses the gait layer and is the easiest way to put a leg where the body cannot support it. Video is a separate MJPEG endpoint so a slow video consumer cannot stall telemetry, and the page falls back to showing camera metadata if no JPEG encoder is installed.

11 Migration to hardware

11.1 What changes

Component	Simulation	Hardware
Joints	MujocoBackend	RobStride02Backend
Camera	SimCameraBackend	NuwaHP60CBackend
Base state	ground truth from the simulator	StateEstimator
Temperature	thermal model	reported, with the model as observer
Launch	backend:=mujoco	backend:=robstride02_can

Nothing else: not a topic, a service, a frame, a message, a controller or a gain.

11.2 Bring-up sequence

1. **Verify the CAN protocol.** Power one motor alone; confirm `GET_DEVICE_ID`, then `ENABLE` and the arrival of feedback frames, then a 0.1 N·m command with $k_p = k_d = 0$ and check the direction of motion. A firmware revision with a different identifier layout changes exactly one file.
2. **Calibrate each joint.** `ros2 run robodog_hardware calibrate_joint` determines `direction` and `offset_rad` against the convention in Section 2.3 and prints a YAML fragment for review. Leave `enable_on_start: false` until signed off.
3. **Identify the estimates.** Rotor inertia from a free-spin deceleration test; joint friction and damping from a swing-down test; thermal constants from a stall at rated torque. Until then the margins in Section 7 carry the uncertainty.
4. **Bring up on a stand,** legs free: joint test sweeps, then named poses, then stance with the body supported.

5. **On the ground**, walk gait first — it is statically stable and keeps three feet down.

11.3 Known gaps

Item	Status	Resolution
CAN frame layout	from the RS02 manual	verify against delivered firmware
Joint direction / offset	unknown	<code>calibrate_joint</code> , per joint
Rotor inertia	estimate 8×10^{-5}	free-spin deceleration
Damping, Coulomb friction	estimate	swing-down test
Thermal R_{th} , C_{th}	estimate	stall at rated torque
Temperature thresholds	estimate	from the above, plus vendor guidance
Camera intrinsics, range	placeholder	datasheet, then <code>camera_calibration</code>
HP60C depth transport	unknown	determine with the device
Joint limits	design values	self-collision check on real geometry
Battery model	nominal voltage only	add a cell model and state of charge

11.4 Planned next steps

The `ros2_control` interface contract is already declared in `urdf/ros2_control.xacro` but not implemented: the runtime is Python.

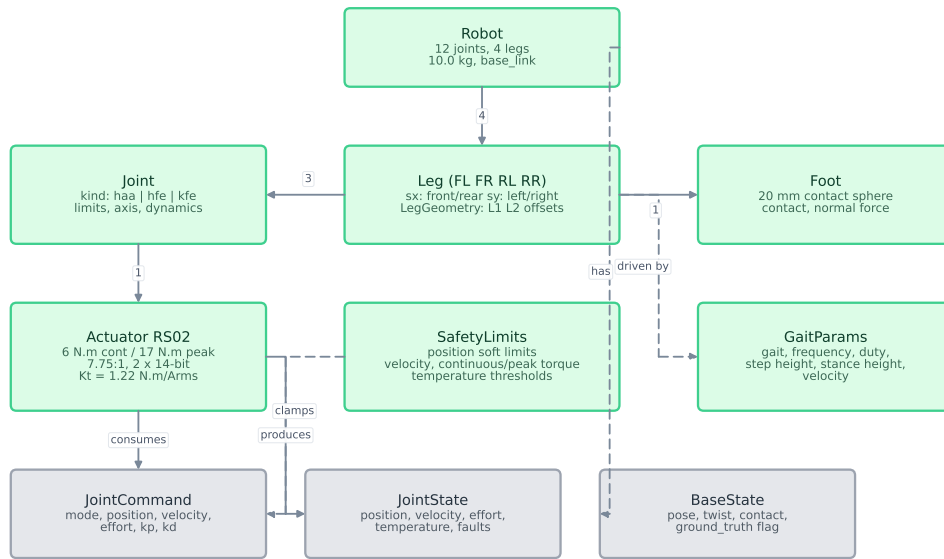
Decision — Python control loop now, C++ `SystemInterface` later

At 400 Hz the measured loop period is 2.49 ms against a nominal 2.50 ms, with worst-case jitter under 1 ms — adequate, and it buys a stack that is fully testable and quick to change during architecture work. The declared `ros2_control` block is the formal statement of the interfaces a C++ `SystemInterface` must provide, and is the migration path once jitter or CPU headroom demands it. Note that the command interface set there is the RS02’s native impedance command, not a single position interface, for the reason in Section 4.

12 Verification

Suite	Tests	What it protects
<code>robodog_description</code>	16	URDF structure, conventions, limits, inertia validity
<code>robodog_hardware</code>	52	CAN codec, backend contract, thermal, bus budget
<code>robodog_control</code>	40	IK exactness, Jacobian, force signs, safety envelope
<code>robodog_sim</code>	46	MJCF vs URDF consistency, world geometry, physics
<code>robodog_perception</code>	17	intrinsics, deprediction, self-occlusion, noise model
Total	171	

The most valuable are the cross-checks: the MJCF’s per-link masses, joint origins, axes and ranges are asserted against the URDF, so the two descriptions cannot drift. Three tests exist specifically because a bug got past a weaker one — the gravity-torque sign, the idle-command rate-limiter seed, and the camera self-occlusion fraction.



Value types crossing the hardware boundary and the entities that own them.

Figure 10: Domain model: the value types that cross the hardware boundary and the entities that own them.

Generated from `robot_parameters.yaml` (CAD 600e4a0e7d58f2e2) and `robstride02.yaml`.
Diagrams: `docs/diagrams/*.drawio`, rendered by `tools/make_diagrams.py`.